

Implementation of JK Flip-Flop

Introduction:

A JK flip-flop is a versatile sequential logic circuit that combines the functionality of both SR (Set-Reset) and T (Toggle) flip-flops while eliminating the invalid state problem. The JK flip-flop has two inputs: J (Set) and K (Reset), along with clock and reset inputs. It operates based on the combination of J and K inputs, where it can hold the previous state, set to 1, reset to 0, or toggle its output. The JK flip-flop is edge-triggered and changes its output only on the positive edge of the clock signal. This makes it one of the most commonly used flip-flops in digital circuit design due to its predictable behavior and elimination of race conditions.

Block diagram

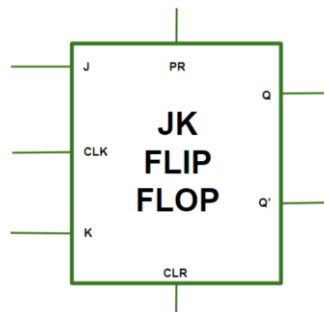


Fig: JK Flip Flop

The JK flip-flop block diagram shows the basic structure with J and K inputs, clock (clk) input, preset, clear, and two outputs Q and $\bar{Q}$ (complement of Q). The flip-flop uses internal logic to determine the next state based on the current J and K input combination. The asynchronous preset forces the output Q to 0 regardless of other inputs when asserted.

Truth table

Preset	Clock(clk)	J	K	Q(next)	$\bar{Q}$ (next)	Operation
1	X	X	X	1	0	Preset
0	↑	0	0	Q	$\bar{Q}$	Hold
0	↑	0	1	0	1	Reset
0	↑	1	0	1	0	Set
0	↑	1	1	$\bar{Q}$	Q	Toggle

here, ↑ indicates positive edge of clock, X indicates don't care condition

Characteristic equation:

$$Q(\text{next}) = J \cdot Q' + K' \cdot Q$$

Application:

JK flip-flops are widely used in digital systems for applications such as binary counters, shift registers, frequency dividers, state machines, memory elements, and as building blocks for more complex sequential circuits. They are particularly useful in counter designs due to their toggle capability.

Procedure to create a lab:

Step 1: Set up Project Directory

- 1.1: Create a directory for your lab: `mkdir <project_name>`
- 1.2: Open lab directory: `cd <Project_name>`
- 1.3: Create directory for project: `mkdir jk_flipflop`
- 1.4: Open project directory: `cd jk_flipflop`

Step 2: Create Design file

- 2.1: Open gvim editor for design with command: `gvim jk_ff.v`

RTL Design Code:

```
module jk_ff (  
    input preset,    // Asynchronous reset (active high)  
    input clk,  
    input j,  
    input k,  
    output reg q,  
    output qn  
);  
assign qn = ~q;  
always @(posedge clk or posedge preset) begin  
    if (preset)  
        q <= 1'b1;          // Reset  
    else begin  
        case ({j, k})  
            2'b00: q <= q;    // Hold  
            2'b01: q <= 1'b0; // Reset  
        endcase  
    end  
end
```

```
2'b10: q <= 1'b1;    // Set
2'b11: q <= ~q;      // Toggle
        endcase
    end
end
endmodule
```

2.2: Save the file using command- :wq!

Step 3: Create Testbench file

3.1: Open gvim editor for testbench with command: gvim jk_ff_tb.v

Testbench:

```
module jk_ff_tb;

    reg preset;        // Asynchronous reset (active high)
    reg clk;
    reg j, k;
    wire q, qn;
    // DUT instantiation
    jk_ff dut (
        .preset(preset),
        .clk    (clk),
        .j      (j),
        .k      (k),
        .q      (q),
        .qn     (qn)
    );

    // Clock generation: 10 ns period
    always #5 clk = ~clk;

    // Monitor the input and output
    initial begin$monitor("%0t|preset=%b j=%b k=%b|q=%b
qn=%b", $time, preset, j, k, q, qn);
```

end

```
initial begin
```

```
    // Initial values
```

```
    clk      = 0;
```

```
    preset = 1;    // Assert async reset
```

```
    j        = 0;
```

```
    k        = 0;
```

```
    // Hold reset for a while
```

```
    #7;           // NOT aligned with clock edge
```

```
    preset = 0;    // Deassert reset (async)
```

```
    // Apply JK combinations
```

```
    #10 j = 0; k = 0;    // Hold
```

```
    #10 j = 0; k = 1;    // Reset
```

```
    #10 j = 1; k = 0;    // Set
```

```
    #10 j = 1; k = 1;    // Toggle
```

```
    #10 j = 1; k = 1;    // Toggle again
```

```
    // Assert async reset in middle of clock cycle
```

```
    #3 preset = 1;
```

```
    #4 preset = 0;
```

```
    #20 $finish;
```

```
end
```

```
// Dump waves
```

```
initial begin
```

```
    $dumpfile("jk_ff_async.vcd");
```

```
    $dumpvars(0, jk_ff_tb);
```

end

endmodule

3.2: Save the testbench file using command : `wq!`

3.3: Check the saved file in current directory: use command : `ls`

Step 4: Simulate the Design with Verilator

4.1: Use the following command to simulate:

```
verilator --binary -j 0 -Wall jk_ff.v jk_ff_tb.v --top
jk_ff_tb --timing --CFLAGS "-std=c++20" --trace
```

4.2: If any error is available, resolve it.

4.3: After simulation a new directory `obj_dir` will be created by tool, check it by using command `ls`

4.4: Open `obj_dir`: `cd obj_dir`

4.5: Use command `make -f Vjk_ff_tb.mk Vjk_ff_tb`

4.6: A new directory with name `Vjk_ff_tb` will be created

4.7: Use command `./Vjk_ff_tb` to compile

4.8: Check output at terminal

Simulation output:

```
0|preset=1 j=0 k=0|q=0 qn=1
0|preset=1 j=0 k=0|q=1 qn=0
7|preset=0 j=0 k=0|q=1 qn=0
27|preset=0 j=0 k=1|q=1 qn=0
35|preset=0 j=0 k=1|q=0 qn=1
37|preset=0 j=1 k=0|q=0 qn=1
45|preset=0 j=1 k=0|q=1 qn=0
47|preset=0 j=1 k=1|q=1 qn=0
55|preset=0 j=1 k=1|q=0 qn=1
60|preset=1 j=1 k=1|q=0 qn=1
60|preset=1 j=1 k=1|q=1 qn=0
64|preset=0 j=1 k=1|q=1 qn=0
65|preset=0 j=1 k=1|q=0 qn=1
75|preset=0 j=1 k=1|q=1 qn=0
- jk_ff_tb.v:40: Verilog $finish
```

The simulation output demonstrates the complete functionality of the JK flip-flop through various test cases. Initially, when `preset=1`, the output `Q` is forced to 0 regardless of `J` and `K` inputs. After reset is released (`preset=0`), the flip-flop responds to different `J, K` combinations:

- `J=0, K=0`: Output holds its previous state
- `J=0, K=1`: Output resets to 0
- `J=1, K=0`: Output sets to 1
- `J=1, K=1`: Output toggles (changes from current state to opposite state)

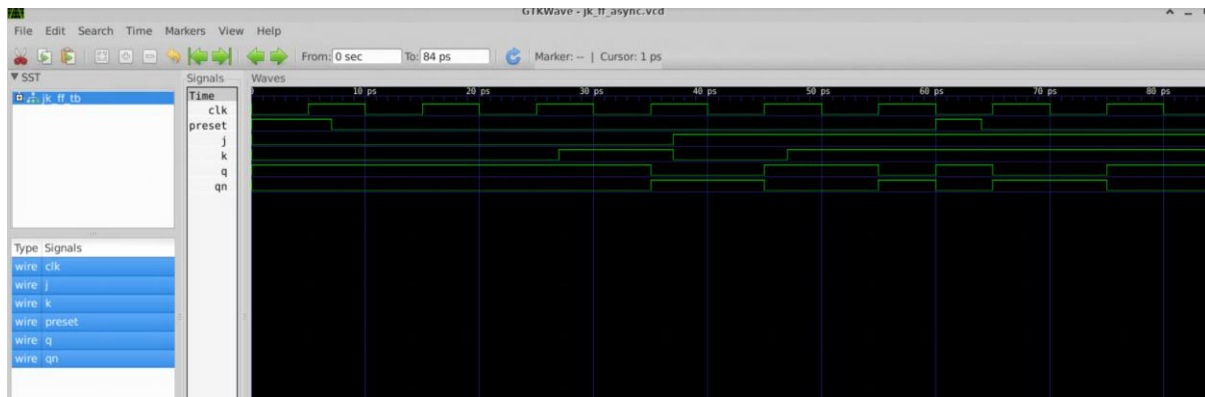
The simulation confirms that the JK flip-flop operates correctly with proper edge-triggered behavior and asynchronous reset functionality.

Step 5: View Waveform Output

5.1: Launch GTKWave to view the waveform file: `gtkwave jk_ff.vcd`

5.2: Analyze the waveform to verify the correct operation of the JK flip-flop

Output waveform:



The output waveform clearly shows the JK flip-flop behavior with distinct phases:

1. **Preset Phase:** $Q=0$ when preset is high, demonstrating asynchronous preset operation
2. **Hold Phase:** Q maintains its state when $J=K=0$
3. **Reset Phase:** Q goes to 0 when $J=0, K=1$
4. **Set Phase:** Q goes to 1 when $J=1, K=0$
5. **Toggle Phase:** Q toggles its state when $J=K=1$, showing the characteristic toggle operation
6. **Hold Phase:** Q maintains its current state when returning to $J=K=0$

The waveform confirms proper clock edge triggering, with all state changes occurring on positive clock edges, and validates the complete functionality of the JK flip-flop design including all four operational modes and asynchronous reset capability.